KIẾN TRÚC MÁY TÍNH

CHƯƠNG 2: KIẾN TRÚC TẬP LỆNH - PHẦN 2

Trình bày các thông tin giới thiệu chung về kiến trúc tập lệnh. Sau đó sẽ đi sâu trình bày về kiến trúc tập lệnh MIPS và cách viết một số chương trình sử dụng các lệnh ASM của MIPS

ThS. Phan Đình Duy

Nội dung môn học

- Chương 1: Những khái niệm và công nghệ máy tính (Tuần 1, 2)
- **Chương 2: Kiến trúc bộ lệnh (Tuần 3, 4, 5, 6)**
- Chương 3: Các phép toán số học trong máy tính (Tuần 7, 8)
- Chương 4: Bộ xử lý (Tuần 10, 11, 12)
- Chương 5: Bộ xử lý pipeline (Tuần 13, 14)
- Ôn tập (Tuần 9, 15)

MỤC TIÊU

1. Trình bày được khái niệm về lệnh, tập lệnh
2. Phân biệt được những nhóm lệnh cơ bản, các loại toán hạng của kiến trúc MIPS
3. Thực hiện chuyển đổi lệnh ngôn ngữ cấp cao sang assembly và mã máy và ngược lại
4. Thực hiện lập trình bằng ngôn ngữ assembly cho MIPS

Ôn tập bài cũ

- Giới thiệu kiến trúc tập lệnh
- Các phép tính - Các lệnh trong MIPS
- Toán hạng (add a, b, c)

NỘI DUNG

1. Giới thiệu
2. Các phép tính - Các lệnh trong MIPS
3. Toán hạng (add a, b, c)
4. **Số có dấu và không dấu**
5. **Các phép tính Logic**
6. **Lập trình ASM trên MARS 4.5 với tập lệnh MIPS**
7. Biểu diễn lệnh
8. Các lệnh điều kiện và nhảy
9. Thủ tục/ hàm

NỘI DUNG

4. Số có dấu và không dấu
5. Các phép tính Logic
6. Lập trình ASM trên MARS 4.5 với tập lệnh MIPS

4. Số có dấu và không dấu

4. Số có dấu và không dấu

- Con người được dạy để suy nghĩ trong hệ cơ số 10, nhưng con số có thể được biểu diễn trong bất kỳ cơ số nào. Ví dụ, 123 cơ số 10 = 1111011 cơ số 2.
- Số lưu trữ trong máy tính như một chuỗi các tín hiệu điện thế cao và thấp, do đó chúng được xem như hệ cơ số 2.
- Ví dụ: Hình vẽ dưới đây cho thấy như thế nào một word của MIPS lưu trữ số 1011:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

(32 bits wide)

- Một word của MIPS có 32 bit, do đó có thê biêu diên các sô từ 0 đên 232−1 (4.294.967.295)
- Bit trọng số nhỏ nhất (The least significant bit – LSB): Bit ngoài cùng bên phải trong một từ nhớ (bit 0)
- Bit trọng số lớn nhất (The most significant bit – MSB): Bit ngoài cùng bên trái trong một từ nhớ (bit 31)

4. Số có dấu và không dấu (tt)

Số dương và âm trong máy tính:

- Các máy tính hiện tại sử dụng bù hai để biểu diễn nhị phân cho số có dấu.
    - Nếu MSB = 0: số dương
    - Nếu MSB = 1: số âm.

⇨Bit thứ 32 (MSB) còn được gọi là bit dấu.

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} = 0_{ten}$
$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} = 1_{ten}$
$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} = 2_{ten}$
. . . . . .

$0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} = 2{,}147{,}483{,}645_{ten}$
$0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} = 2{,}147{,}483{,}646_{ten}$
$0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} = 2{,}147{,}483{,}647_{ten}$
$1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} = -2{,}147{,}483{,}648_{ten}$
$1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} = -2{,}147{,}483{,}647_{ten}$
$1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} = -2{,}147{,}483{,}646_{ten}$
. . . . . .

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} = -3_{ten}$
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} = -2_{ten}$
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} = -1_{ten}$

4. Số có dấu và không dấu (tt)

- Nửa phần dương của các con số, từ 0 đến $2{,}147{,}483{,}647_{ten}$ ($2^{31} - 1$), biểu diễn như thường.
- Phần số âm biểu diễn:
  - $1000\ldots0000_{two}$ = -2,147,483,648ten
  - $1000\ldots0001_{two}$ = -2,147,483,647ten
  - $1111\ldots1111_{two}$ = -1ten

$$
\begin{array}{lcl}
0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} & = & 0_{ten} \\
0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} & = & 1_{ten} \\
0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} & = & 2_{ten} \\
\ldots & & \ldots \\
0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} & = & 2{,}147{,}483{,}645_{ten} \\
0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} & = & 2{,}147{,}483{,}646_{ten} \\
0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} & = & 2{,}147{,}483{,}647_{ten} \\
1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} & = & -2{,}147{,}483{,}648_{ten} \\
1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} & = & -2{,}147{,}483{,}647_{ten} \\
1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} & = & -2{,}147{,}483{,}646_{ten} \\
\ldots & & \ldots \\
1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} & = & -3_{ten} \\
1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} & = & -2_{ten} \\
1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} & = & -1_{ten}
\end{array}
$$

- Bù hai có một số âm $-2{,}147{,}483{,}648_{ten}$, mà không có số dương tương ứng.

4. Số có dấu và không dấu (tt)

- Công thức chuyển từ một số bù hai sang số hệ 10:

$$(x31 \times -2^{31}) + (x30 \times 2^{30}) + (x29 \times 2^{29}) + \ldots + (x1 \times 2^{1}) + (x0 \times 2^{0})$$

- **Lưu ý:** Bit dấu được nhân với -2^{31}, và phần còn lại của các bit sau đó được nhân với các số dương của các giá trị cơ số nào tương ứng của chúng.
- **Ví dụ: đổi từ hệ 2 sang hệ 10**

$$1111\ \ 1111\ \ 1111\ \ 1111\ \ 1111\ \ 1111\ \ 1111\ \ 1100_{two}$$

$$\begin{aligned}
&(1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \ldots + (1 \times 2^{2}) + (0 \times 2^{1}) + (0 \times 2^{0}) \\
&= -2^{31} \quad + \quad 2^{30} \quad + \quad 2^{29} \quad + \ldots + \quad 2^{2} \quad + \quad 0 \quad + \quad 0 \\
&= -2{,}147{,}483{,}648_{ten} + 2{,}147{,}483{,}644_{ten} \\
&= -4_{ten}
\end{aligned}$$

4. Số có dấu và không dấu (tt)

Mở rộng số có dấu:

- Làm thế nào để chuyển đổi một số nhị phân được biểu diễn trong n bit thành một số biểu diễn với nhiều hơn n bit?
- Ví dụ: Chuyển đổi số nhị phân 16 bit của số 2_{ten} và -2_{ten} thành số nhị phân 32 bit.
- 2_{ten}: $0000\ 0000\ 0000\ 0010_{two}$ → $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two}$
- -2_{ten}: $1111\ 1111\ 1111\ 1110_{two}$ → $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two}$

4. Số có dấu và không dấu (tt)

Khi làm việc với các lệnh của MIPS, lưu ý:

- Mở rộng có dấu (Sign-extend)
- Mở rộng không dấu (Zero-extend)

5. Các phép tính Logic

Các phép tính Logic

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR	|	|	or, ori

C và Java các phép tính logic và lệnh MIPS tương ứng.

- Shift: Lệnh dịch chuyển bit.
- AND: là phép toán logic “VÀ”.
- OR: là một phép toán logic “HOẶC”
- NOR: NOT OR.
- Hằng số rất hữu ích trong các phép toán logic AND và OR cũng như trong phép tính số học, vì vậy MIPS cung cấp các lệnh trực tiếp andi và ori.

Các phép tính Logic - Lệnh shift

Giả sử thanh ghi s1 chứa giá trị nhị phân 8 bit 00000**110** (6_{10})

- Sau khi thực hiện câu lệnh sll $t0, $s1, 1 thì giá trị thanh ghi

  t0 = 0000**110**0 (12_{10})

- Sau khi thực hiện câu lệnh sll $t1, $s1, 2 thì giá trị thanh ghi

  t1 = 000**110**00 (24_{10})

- Sau khi thực hiện câu lệnh srl $t2, $s1, 1 thì giá trị thanh ghi

  t2 = 00000**11** (3_{10})

Các phép tính Logic - Lệnh or

- Cho giá trị các thanh ghi như sau:
  - $t0 = 0x55555555
  - $t1 = 0x12345678
- Hãy cho biết giá trị của thanh ghi $t2 sau khi chạy các lệnh sau:

sll $t2, $t0, 4

or $t2, $t2, $t1

Các phép tính Logic - Lệnh or

- Cho giá trị các thanh ghi như sau:
  - $t0 = 0x55555555 = 01010101010101010101010101010101
  - $t1 = 0x12345678 = 00010010001101000101011001111000
- Hãy cho biết giá trị của thanh ghi $t2 sau khi chạy các lệnh sau:

sll $t2, $t0, 4 => *$t2* = 01010101010101010101010101010000

or $t2, $t2, $t1 với $t1 = 00010010001101000101011001111000

=> $t2 = 01010111011110101010101111011111000

Các phép tính Logic - Lệnh andi

- Cho giá trị các thanh ghi như sau:
  - $t0 = 0x55555555
- Hãy cho biết giá trị của thanh ghi $t2 sau khi chạy các lệnh sau:

srl $t2, $t0, 3

andi $t2, $t2, 0xFFEF

Các phép tính Logic - Lệnh andi

- Cho giá trị các thanh ghi như sau:
  - $t0 = 0x55555555 = 01010101010101010101010101010101
- Hãy cho biết giá trị của thanh ghi $t2 sau khi chạy các lệnh sau:

srl $t2, $t0, 3 => $t2 = 00001010101010101010101010101010

andi $t2, $t2, 0xFFEF = 0000 0000 0000 0000 1111 1111 1110 1111

- => $t2 = 00001010101010101010101010101010

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}

Các phép tính Logic - Bài tập

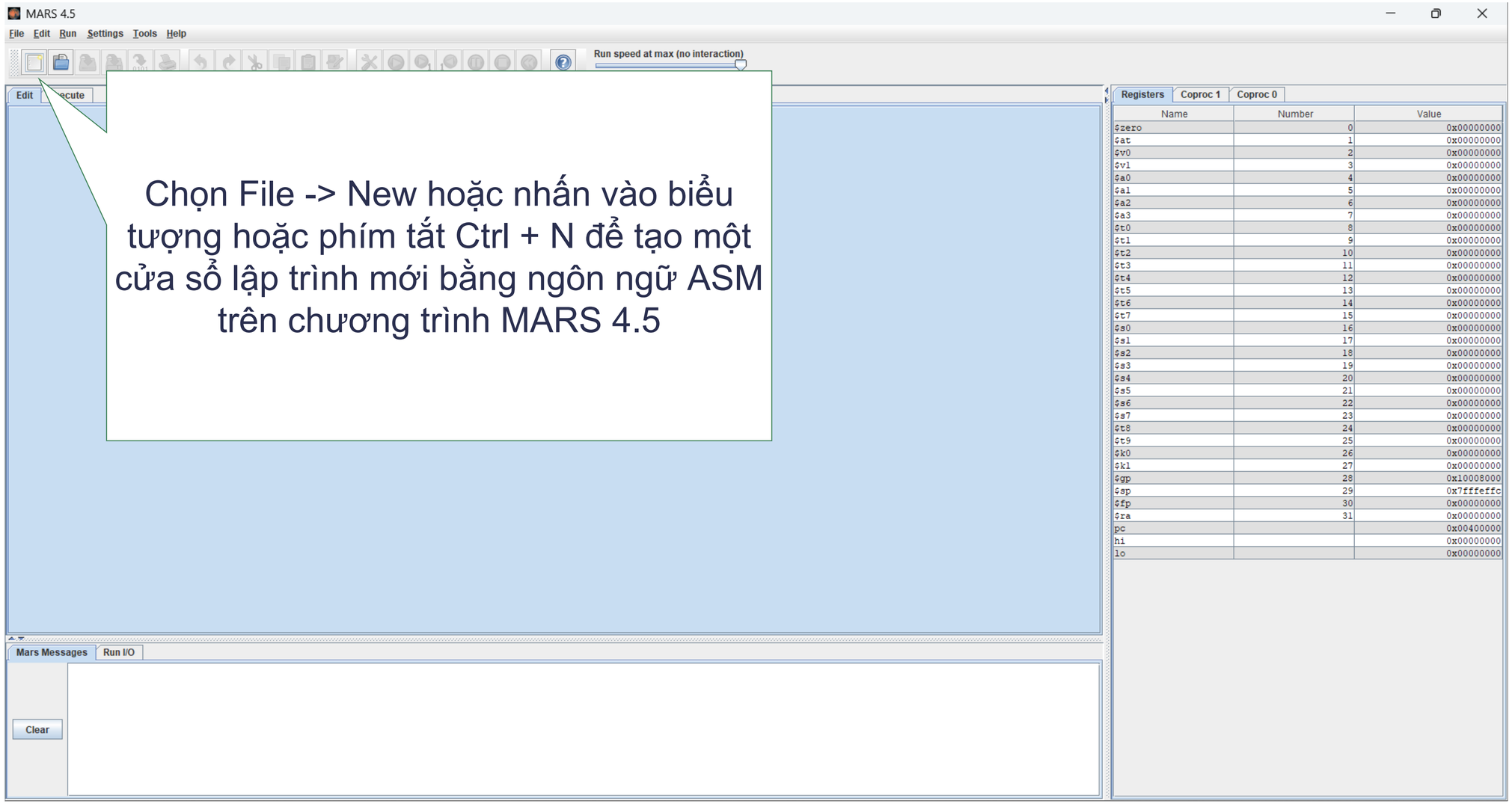
- Cho giá trị các thanh ghi như sau:
  - $t0 = 0x20232024
- Hãy cho biết giá trị của thanh ghi $thjg sau khi chạy các lệnh sau:

sll $s1, $t0 3

ori $t2, $s1, 0x2025

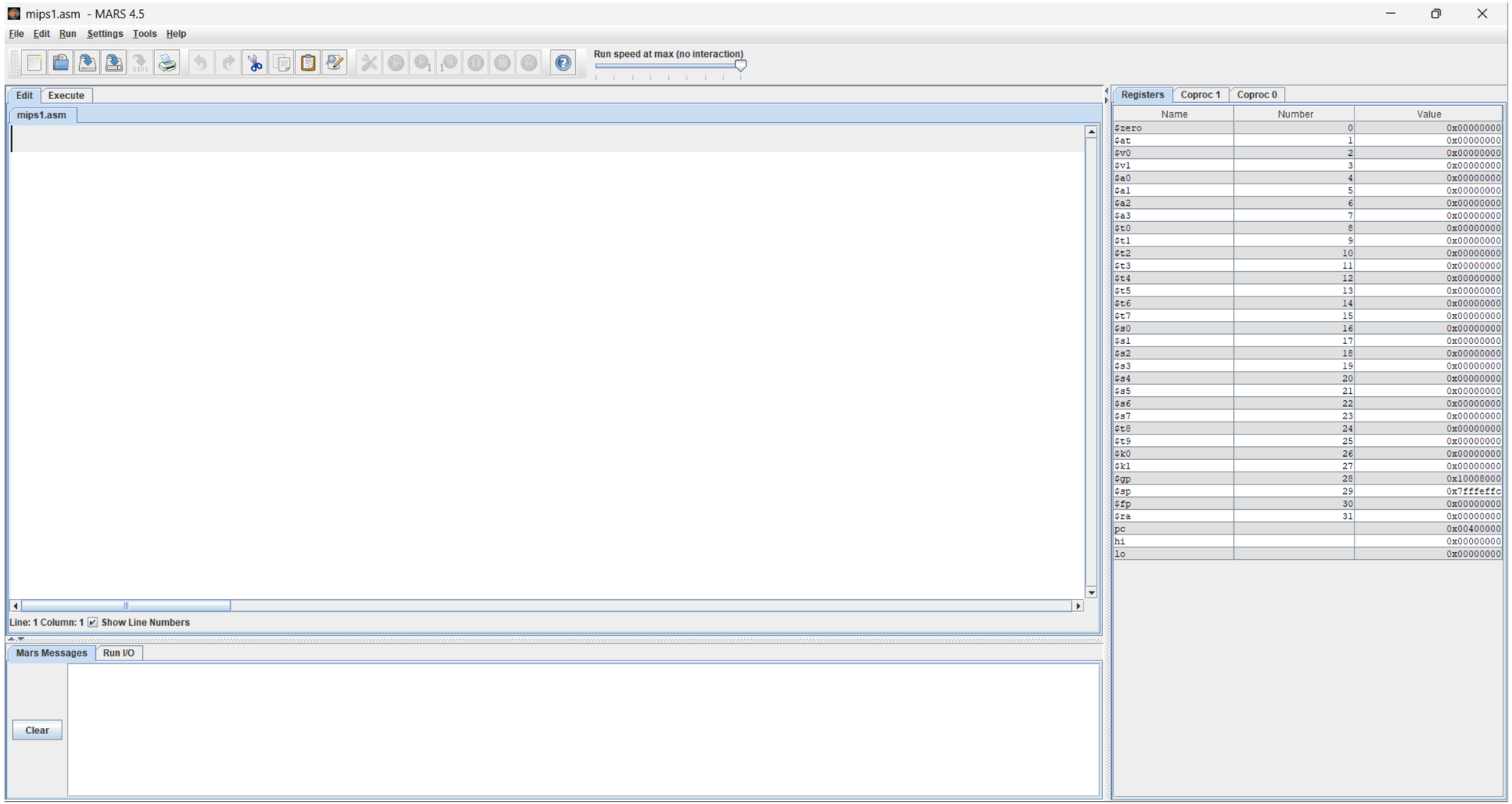
6. Lập trình ASM trên MARS 4.5

6. Lập trình ASM trên MARS 4.5



Chọn File -> New hoặc nhấn vào biểu tượng hoặc phím tắt Ctrl + N để tạo một cửa sổ lập trình mới bằng ngôn ngữ ASM trên chương trình MARS 4.5

6. Lập trình ASM trên MARS 4.5 (tt)



Cấu trúc chương trình ASM trên MARS 4.5

Chú thích bằng đầu bằng ký tự #

```
        .data       # Khai báo dữ liệu ngay sau hàng này
                    # Khai báo dữ liệu
        .text       # Viết chương trình ngay sau hàng này
main:               # Nhãn thể hiện bắt đầu chương trình
                    # Viết chương trình
```

Khai báo dữ liệu

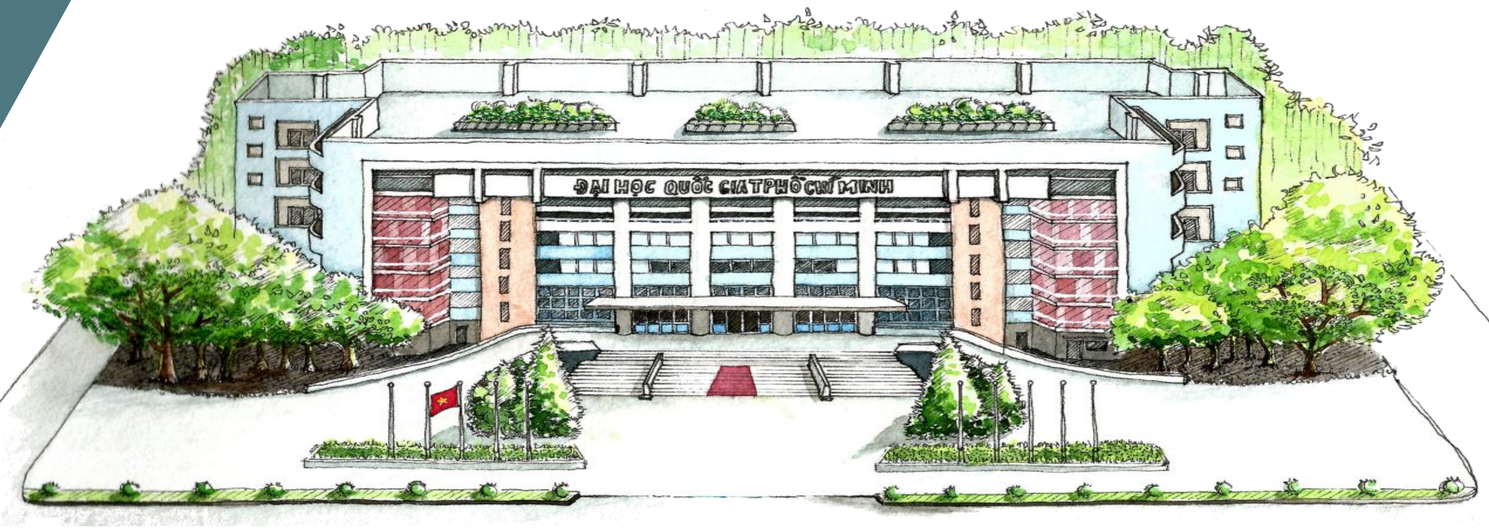
- Ký số: Hệ thập phân (17), Hệ thập lục phân thêm tiền tố 0x (0x17)
- Ký tự: Đặt trong cặp nháy đơn (‘c’)
- Chuỗi: Đặt trong cặp nháy kép (“IT006”)

[<nhãn>:] .<kiểu dữ liệu> <danh sách giá trị>

- Ví dụ:
  - var1: .word 3 # biến nguyên var1 có kích thước 1 word
  - arr1: .byte ‘a’, ‘k’ # mảng arr1 có 2 phần tử, mỗi phần tử 1 byte
  - arr2: .space 40 # mảng arr2 có kích thước 40 byte liên tục
  - str1: .asciiz “ahihi” # chuỗi str1 có kết thúc bởi null

6. Thực hành lập trình ASM trên MARS 4.5

Tóm tắt lại nội dung buổi học

- Số có dấu và không dấu
- Các phép tính Logic
- Lập trình ASM trên MARS 4.5 với tập lệnh MIPS

THẢO LUẬN